

로봇 추격

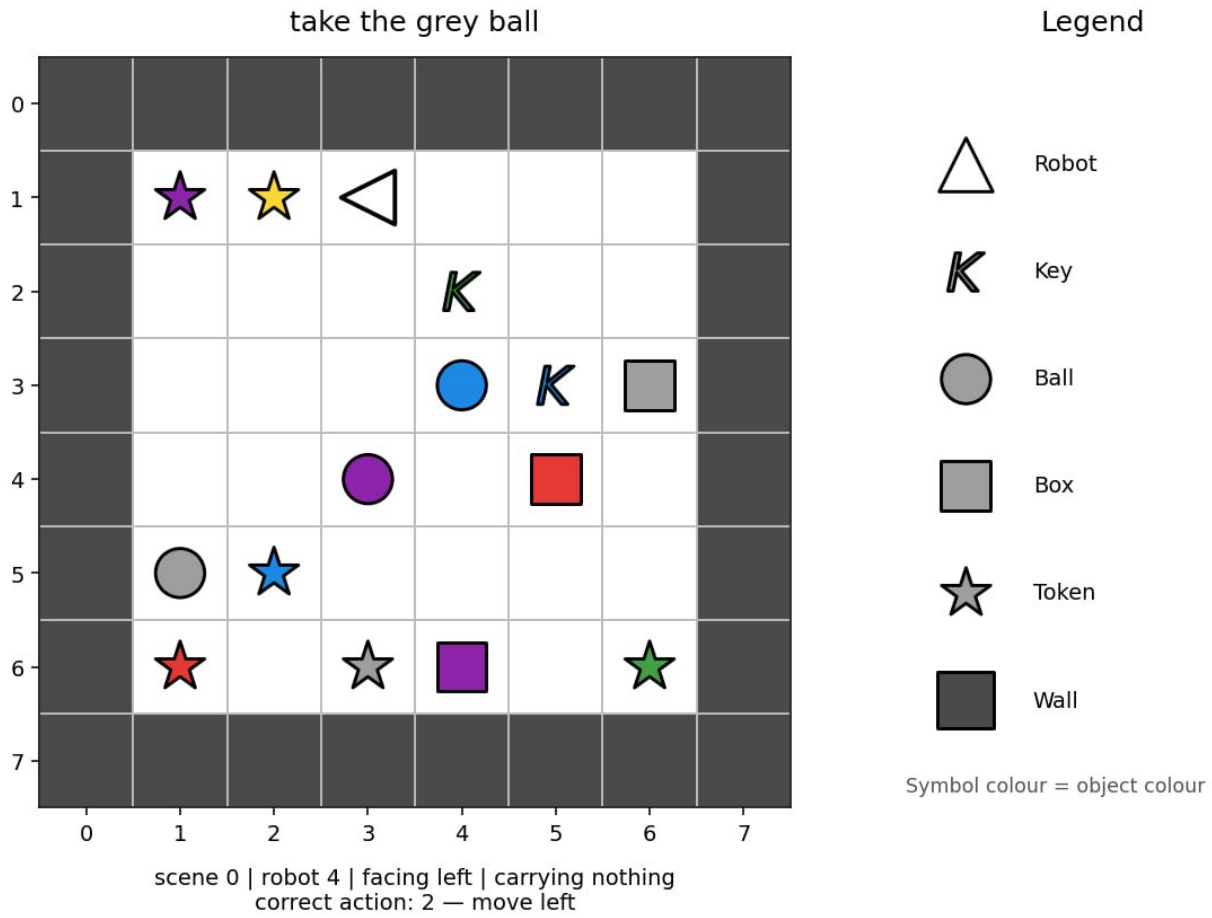
- 제한 시간: 5분
- 환경: GPU 1개(약 16 GB VRAM), 인터넷 없음
- 솔루션 크기: `solution.ipynb` ≤ 1 MB
- 저장 공간: 5 GB

과제

로봇이 여섯 대 있습니다. 각 로봇은 그리드(격자)로 표현된 작은 방 안에서 작동합니다. 각 방에는 벽으로 둘러싸인 6×6 크기의 플레이 (돌아다닐 수 있는)가능 영역이 있으므로, 전체 `image` 배열의 크기는 8×8 입니다(플레이 가능 영역 + 벽).

각 로봇은 과제를 설명하는 영어 지시문을 받습니다. 스냅샷은 로봇이 과제를 수행하는 도중 어느 시점에서든 촬영될 수 있습니다. 목표는 로봇의 다음 행동을 예측하는 것입니다.

로봇이 항상 최단 경로를 따르는 것은 아닙니다. Robot 0은 Robot 1과 다르게 행동할 수 있지만, 각 로봇은 자신만의 일관된 패턴을 따릅니다. 올바른 다음 행동이 포함된 학습 예제를 활용하여 이러한 패턴을 학습하십시오.



미션에는 세 가지 유형이 있습니다:

- 물체로 **가기(go to)**, 예를 들어 "approach the red ball";
- 물체를 **집어 들기(pick up)**, 예를 들어 "grab the blue key";
- 한 물체를 다른 물체 옆에 **놓기(put one object next to another)**, 예를 들어 "place the red box beside the green ball".

같은 지시문이 여러 방식으로 표현될 수 있습니다. 테스트 세트에는 익숙한 표현, 색상, 물체 유형으로 구성된 새로운 조합이 포함될 수 있습니다. 테스트 세트에서 사용되는 모든 단어, 표현 패턴, 색상, 물체 유형, 미션 유형은 학습 세트에도 등장합니다.

각 샘플은 다음 필드를 가집니다:

필드	의미
robot_id	6대의 로봇 중 어느 것인지(0-5)
image	방을 나타내는 $8 \times 8 \times 2$ 정수 배열로, 채널 0은 범주형 object_idx(예: 1=empty, 2=wall, 10=robot)를, 채널 1은 범주형 colour_idx(0-5)를 담습니다.
direction	로봇이 현재 바라보는 방향
mission	눈에 보이는 자연어 지시문

필드	의미
carrying	들고 있는 물체에 대한 null 또는 [object_idx, colour_idx]

각 행은 무작위 순서로 배열된 독립적인 스냅샷입니다. 이들은 에피소드를 구성하지 않으며 (연속적으로 나열된 것이 아님), 평가 시점에 이전 관측이나 행동은 제공되지 않습니다.

제공된 `visualize_dataset.ipynb`을 통해 다양한 상황에서 모델이 사용할 수 있는 관측을 살펴볼 수 있습니다.

그리드 인코딩

`image[row][column] = [object_idx, colour_idx]`. 첫 번째 인덱스는 위에서 아래로의 행이고, 두 번째 인덱스는 왼쪽에서 오른쪽으로의 열입니다. 배열에는 외벽 경계가 포함되어 있으므로, 이동 가능한 내부 영역은 6×6입니다.

물체 id:

id	물체
1	빈 칸
2	벽
5	열쇠
6	공
7	상자
10	로봇
11	토كن

토кен은 방 안에 나타날 수 있으나 미션에서 언급되는 일은 없습니다.

색상 id는 0 빨강, 1 초록, 2 파랑, 3 보라, 4 노랑, 5 회색입니다. 색상 채널은 빈 칸과 벽에 대해서는 아무 의미가 없습니다.

이미지에는 위의 두 채널만 있습니다. 로봇의 방향은 최상위 `direction` 필드에 한 번만 제공되며, `image` 내부에 중복되어 담기지 않습니다.

행동

코드 0-3에 대해, 이동 행동은 다음의 절대적 매핑을 사용합니다:

행동	의미
0	위로 이동
1	아래로 이동
2	왼쪽으로 이동
3	오른쪽으로 이동
4	집어 들기
5	내려놓기

`direction` 필드는 현재 바라보는 방향을 다음과 같이 나타냅니다: 0 = 위(`row - 1`), 1 = 아래(`row + 1`), 2 = 왼쪽(`col - 1`), 3 = 오른쪽(`col + 1`).

이동 행동은 먼저 로봇을 해당 절대 방향으로 돌린 다음, 한 칸 이동을 시도합니다. 벽이나 물체가 이동을 막을 수 있지만, 방향은 그래도 변경됩니다. `pick up`과 `drop`는 방향에 의해 정의되는 인접한 대상 칸에만 작용합니다 (예: `direction=0`이면 (`row - 1, col`)에 있는 물체에 대해 동작 합니다).

데이터셋

두 개의 폴더가 제공됩니다:

폴더	행 수	labels.json?	용도
dataset/train/	60,000	포함됨	모델 학습
dataset/test_public/	3,600	개발용 사본에 포함됨	파이프라인 실행 및 자체 채점

각 폴더에는 위에서 설명한 샘플들의 JSON 리스트인 `observations.json`이 들어 있습니다. `labels.json`은 행동(0-5)의 정렬된 JSON 리스트입니다.

학습 세트에는 로봇별로 정확히 10,000행이 있고, 각 과제 계열(task family)마다 20,000행이 있습니다. 공개 테스트에는 로봇별로 600행이 있습니다. 배열이 필요하다면 `image`을 `numpy.asarray(...)`으로 감싸십시오.

채점 시에는 `dataset/test_public/`가 동일한 형식이지만 `labels.json`이 없는 비공개 3,600개 관측 세트로 투명하게 대체됩니다. 공개 리더보드는 `test_leaderboard_a`를 사용하고, 최종 순위는 `test_leaderboard_b`를 사용합니다. 테스트 레이블을 무조건 읽는 노트북은 실패합니다. 레이블은 `dataset/train/`에서만 읽으십시오.

출력

노트북의 `작업` 디렉터리에 `predictions.json`을 작성하십시오. 이 파일은 `dataset/test_public/observations.json`의 각 행에 대해 정수 행동(0-5) 하나를 같은 순서로 담은

JSON 리스트여야 합니다. 여섯 개의 샘플을 포함하는 가상의 테스트 세트에 대해, 유효한 출력은 다음과 같습니다:

```
[0, 3, 2, 2, 5, 4]
```

JSON 파일이 없거나 유효하지 않은 경우, 예측 개수가 잘못된 경우, 정수가 아닌 값이 있는 경우, 또는 {0,1,2,3,4,5} 같이 정상적인 범위를 벗어난 행동이 있는 경우에는 점수 없이 거부됩니다.

채점

채점은 0-100 척도의 **로봇별 평균 정확도(mean per-robot accuracy)**입니다. 먼저 각 로봇에 대해 독립적으로 정확도를 계산하고, 그다음 여섯 로봇 전체에 대해 평균을 냅니다. 따라서 모든 로봇은 동일한 가중치를 가집니다.

제출 방법

1. `solution.ipynb`을 열고 모든 셀을 실행하십시오.
2. 공개 테스트 세트에 대한 3,600개의 예측이 담긴 `predictions.json`을 작성하는지 확인하십시오.
3. 원한다면 모델을 개선하십시오. 제공된 베이스라인은 요구되는 입력 및 출력 형식만을 보여줍니다.
4. JupyterLab Git 탭에서 `solution.ipynb`을 스테이징하고 커밋한 다음 push하십시오.
5. Contest 페이지로 돌아가 **Submit**을 클릭하십시오.

`solution.ipynb`이라는 이름의 파일을 정확히 하나만 제출하십시오.